

C++ UNIT- I

1. List any four key differences between C and C++.

A)

S.No	C	C++
1	C is a procedural programming language .	C++ is an object-oriented programming language (OOP) .
2	Focuses on functions and procedures .	Focuses on objects and classes .
3	Does not support classes and objects .	Supports classes and objects .
4	Does not support inheritance .	Supports inheritance , enabling code reuse.
5	Does not support polymorphism .	Supports polymorphism (function and operator overloading).
6	Data is less secure because there is no data hiding.	Provides data security through encapsulation and data hiding.
7	Uses standard input/output functions like scanf() and printf().	Uses stream objects like cin and cout for input/output.
8	Does not support constructors and destructors .	Supports constructors and destructors for object initialization and cleanup.

Conclusion

C is mainly used for **procedural programming** and system-level applications, whereas C++ extends C by adding **object-oriented features**, making it suitable for developing large, secure, and reusable software applications.

2. List and briefly explain key concepts of Object-Oriented Programming.

A) Object-Oriented Programming (OOP)

Object-Oriented Programming (OOP) is a programming paradigm that organizes programs using **objects** and **classes**. It helps in developing secure, reusable, and maintainable software.

Key Concepts of OOP

1. Class

A **class** is a blueprint or template for creating objects. It defines the data members (attributes) and member functions (methods) of an object.

Example: Student, Car, BankAccount

2. Object

An **object** is an instance of a class. It represents a real-world entity and can access the data and functions defined in the class.

Example: student1, car1

3. Encapsulation

Encapsulation is the process of wrapping data (variables) and methods (functions) into a single unit (class). It protects data from unauthorized access using access specifiers like private, public, and protected.

Advantage: Improves data security.

4. Abstraction

Abstraction means hiding unnecessary implementation details and showing only the essential features to the user.

Example: An ATM allows users to withdraw money without knowing its internal working.

5. Inheritance

Inheritance is the process by which one class acquires the properties and behaviors of another class.

Advantage: Promotes code reusability and reduces duplication.

Example: A Car class inherits features from a Vehicle class.

6. Polymorphism

Polymorphism means "one interface, many forms." It allows the same function or operator to perform different tasks.

Types:

- Compile-time Polymorphism (Function Overloading, Operator Overloading)
- Run-time Polymorphism (Function Overriding)

7. Dynamic Binding

Dynamic Binding means the method to be executed is decided during **runtime** rather than compile time.

Advantage: Supports runtime polymorphism.

8. Message Passing

Message Passing is the process of communication between objects by calling each other's methods.

Example: One object sends a request to another object to perform a task.

PROGRAM:

```
#include <iostream>

using namespace std;

class Student
```

```

{
public:
    string name;
    int age;
    void getData()
    {
        cout << "Enter Name: ";
        cin >> name;
        cout << "Enter Age: ";
        cin >> age;
    }
    void display()
    {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
    }
};

int main()
{
    Student s1; // Object creation
    s1.getData(); // Calling member function
    s1.display();
    return 0;
}

```

OUTPUT:

Enter Name: Akhil

Enter Age: 18

Name: Akhil

Age: 18

3. Explain any four disadvantages of conventional (procedure-oriented) programming.

A) Procedure-Oriented Programming (POP)

Procedure-Oriented Programming (POP) is a programming approach where the program is divided into functions or procedures. Although suitable for small programs, it has several disadvantages.

Any Four Disadvantages of POP

1. No Data Security

- Data is generally shared among functions.
- There is no data hiding, so unauthorized access or modification is possible.

2. Difficult to Manage Large Programs

- As the program size increases, it becomes difficult to understand, debug, and maintain.
- Changes in one function may affect other parts of the program.

3. No Code Reusability

- POP does not support inheritance.
- Existing code cannot be reused efficiently, leading to code duplication.

4. No Real-World Modeling

- POP focuses on functions rather than objects.
- It cannot represent real-world entities such as students, cars, or bank accounts effectively.

5. Difficult Maintenance

- Updating or modifying large procedural programs is time-consuming and error-prone.

6. No Encapsulation

- Data and functions are separate.
- This reduces security and makes programs harder to organize.

7. No Polymorphism

- POP does not support function overriding or operator overloading.
- Flexibility is limited compared to OOP.

8. High Complexity

- Large procedural programs become complex because functions are highly dependent on shared data.

Conclusion

Conventional (Procedure-Oriented) Programming is suitable for **small applications**, but it has limitations such as **poor security, low code reusability, difficult maintenance, and inability to model real-world objects**. These disadvantages led to the development of **Object-Oriented Programming (OOP)**.

4. Describe the concept of "data abstraction" with respect to OOP

A) Data Abstraction is one of the most important concepts of **Object-Oriented Programming (OOP)**. It is the process of **hiding the internal implementation details** of an object and **showing only the essential features** to the user.

Real-Life Example

An **ATM Machine** is a good example of data abstraction.

- The user enters the PIN and withdraws money.
- The user does not know how the ATM verifies the PIN or processes the transaction internally.
- Only the required operations are visible.

Advantages of Data Abstraction

1. Hides implementation details.
2. Improves data security.
3. Reduces program complexity.
4. Makes the program easy to maintain.
5. Allows changes in implementation without affecting users

PROGRAM:

```
#include <iostream>

using namespace std;

class Student
{
private:
    int marks;

public:
    void setMarks(int m)
    {
        marks = m;
    }

    void displayMarks()
    {
        cout << "Marks = " << marks << endl;
    }
};

int main()
{
    Student s;
```

```
s.setMarks(90);

s.displayMarks();

return 0;

}
```

OUTPUT:

Marks = 90

Explanation

- marks is declared as **private**, so it cannot be accessed directly from outside the class.
- The user accesses it only through the **public** functions setMarks() and displayMarks().
- The internal data is hidden, which demonstrates **Data Abstraction**.

5. Write a CPP program to demonstrate the scope resolution operator to access a global variable when a local variable with the same name exists.

A) The **Scope Resolution Operator (::)** is used to access a **global variable** when a **local variable** has the same name. It tells the compiler to use the global variable instead of the local one.

PROGRAM:

```
#include <iostream>

using namespace std;

// Global variable

int num = 100;

int main()

{

    // Local variable

    int num = 50;

    cout << "Local Variable = " << num << endl;

    cout << "Global Variable = " << ::num << endl;

    return 0;

}
```

OUTPUT:

Local Variable = 50

Global Variable = 100

Explanation

- `int num = 100;` is a **global variable**.
- `int num = 50;` is a **local variable** inside `main()`.
- When `num` is used, the compiler accesses the **local variable**.
- When `::num` is used, the **scope resolution operator (::)** accesses the **global variable**.

Advantages of Scope Resolution Operator

1. Accesses global variables when local variables have the same name.
2. Avoids naming conflicts.
3. Improves code readability.
4. Helps distinguish between local and global variables.

6. Analyze how namespaces help in avoiding name conflicts in large C++ explain with code snippet.

A) A **namespace** in C++ is a feature used to **group related classes, functions, and variables** under a unique name. It helps avoid **name conflicts** when different libraries or programmers use the same identifier.

Syntax

```
namespace NamespaceName  
{  
    // declarations  
}
```

Need for Namespaces

In large C++ projects:

- Different programmers may create functions or variables with the same name.
- Multiple libraries may contain identical function names.
- This creates **name conflicts (ambiguity)**.

Namespaces solve this problem by placing identifiers in separate scopes.

Advantages of Namespaces

1. Avoids name conflicts between different libraries.
2. Organizes code into logical groups.
3. Improves readability and maintainability.

4. Makes large projects easier to manage.

PROGRAM:

```
#include <iostream>

using namespace std;

namespace First
{
    int value = 10;
}

namespace Second
{
    int value = 20;
}

int main()
{
    cout << "First Namespace Value = " << First::value << endl;
    cout << "Second Namespace Value = " << Second::value << endl;
    return 0;
}
```

OUTPUT:

First Namespace Value = 10

Second Namespace Value = 20

Explanation

- Two namespaces, **First** and **Second**, both contain a variable named value.
- Normally, having two variables with the same name would cause a conflict.
- By using the **scope resolution operator (::)**, we specify the namespace:
 - First::value → accesses value in the First namespace.
 - Second::value → accesses value in the Second namespace.
- Thus, namespaces allow the same identifier to exist without ambiguity.

7. List and explain the uses of Scope Resolution Operator with code snippet of each one.

A) The **Scope Resolution Operator (::)** in C++ is used to specify the scope of a variable, function, or class member. It helps the compiler identify which variable or function should be accessed.

Uses of Scope Resolution Operator

1. Accessing a Global Variable

When a local variable has the same name as a global variable, :: is used to access the global variable.

Program

```
#include <iostream>

using namespace std;

int x = 100; // Global variable

int main()
{
    int x = 50; // Local variable

    cout << "Local x = " << x << endl;

    cout << "Global x = " << ::x << endl;

    return 0;
}
```

Output

Local x = 50

Global x = 100

2. Defining Class Member Functions Outside the Class

The :: operator is used to define member functions outside the class.

Program

```
#include <iostream>

using namespace std;

class Student
{
public:
    void display();
}
```

```
};

void Student::display()

{
    cout << "Hello from Student Class";
}

int main()

{
    Student s;

    s.display();

    return 0;
}
```

Output

Hello from Student Class

3. Accessing Members of a Namespace

The :: operator is used to access variables or functions inside a namespace.

Program

```
#include <iostream>

using namespace std;

namespace Demo

{
    int value = 25;
}

int main()

{
    cout << "Value = " << Demo::value;

    return 0;
}
```

Output

Value = 25

4. Accessing Static Class Members

The :: operator is used to access static data members or static member functions.

Program

```
#include <iostream>

using namespace std;

class Test
{
public:
    static int count;
};

int Test::count = 10;

int main()
{
    cout << "Count = " << Test::count;

    return 0;
}
```

Output

Count = 10

Summary Table

Use	Purpose
Global Variable	Access global variable when local variable has the same name
Class Member Function	Define member functions outside the class
Namespace	Access variables/functions inside a namespace
Static Members	Access static data members and static member functions

8. Write a C++ program to demonstrate default arguments.

A) Default arguments are values assigned to function parameters at the time of function declaration. If the caller does not provide an argument, the function uses the default value automatically.

PROGRAM:

```
#include <iostream>

using namespace std;
```

```
// Function with default arguments

void display(string name = "Student", int age = 18)
{
    cout << "Name: " << name << endl;
    cout << "Age : " << age << endl;
    cout << endl;
}

int main()
{
    display();          // Uses both default values
    display("Akhil");   // Uses default age
    display("Rahul", 20); // Uses both given values
    return 0;
}
```

OUTPUT:

Name: Student

Age : 18

Name: Akhil

Age : 18

Name: Rahul

Age : 20

Explanation

- The function display() has two default arguments:
 - name = "Student"
 - age = 18
- display(); → Uses both default values.
- display("Akhil"); → Uses "Akhil" for name and the default value 18 for age.
- display("Rahul", 20); → Uses the values provided in the function call.

Explanation

- The function display() has two default arguments:

- name = "Student"
- age = 18
- display(); → Uses both default values.
- display("Akhil"); → Uses "Akhil" for name and the default value 18 for age.
- display("Rahul", 20); → Uses the values provided in the function call.

9. List the Advantages of OOP

A) Object-Oriented Programming (OOP) is a programming paradigm that organizes programs into **objects**, which contain **data (attributes)** and **functions (methods)**. OOP makes software more organized, reusable, and secure.

Advantages of OOP

1. Code Reusability

- OOP allows code reuse through **inheritance**.
- Existing classes can be reused to create new classes, reducing development time.

2. Data Security (Encapsulation)

- Data is hidden from unauthorized access using **private** and **protected** access specifiers.
- Access is provided only through public member functions.

3. Data Abstraction

- Only essential information is shown to the user, while internal implementation details are hidden.
- This makes programs easier to use and maintain.

4. Modularity

- Programs are divided into small, independent classes.
- Each class performs a specific task, making the program easier to understand and manage.

5. Easy Maintenance

- Changes made in one class usually do not affect other classes.
- This simplifies debugging and updating the program.

6. Flexibility (Polymorphism)

- The same function name can perform different tasks using **function overloading** or **function overriding**.
- This increases program flexibility.

7. Improved Readability

- OOP organizes code into classes and objects, making it more structured and easier to read.

8. Easy Debugging

- Since the program is divided into classes, errors can be found and fixed more easily.

9. Scalability

- New features or classes can be added without modifying the entire program.
- Suitable for developing large software applications.

10. Real-World Modeling

- OOP represents real-world entities such as **Student, Car, Bank Account, Mobile Phone**, etc., making programs more realistic and easier to design.

Summary Table

Advantage	Description
Code Reusability	Reuse existing code through inheritance
Data Security	Protect data using encapsulation
Data Abstraction	Hide implementation details
Modularity	Divide program into independent classes
Easy Maintenance	Easy to update and modify
Flexibility	Supports polymorphism
Improved Readability	Well-organized and structured code
Easy Debugging	Errors are easier to identify and fix
Scalability	Easy to add new features
Real-World Modeling	Models real-life objects effectively

